

Arbres Binaires de Recherche

ABR - Rouge-Noir - AVL

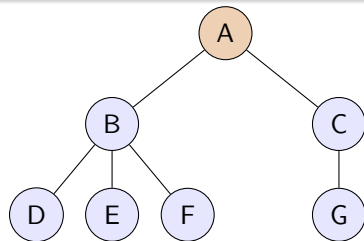
Plan du cours

1. Introduction aux Arbres
2. Arbre Binaire
3. Parcours d'un Arbre Binaire
4. Arbre Binaire de Recherche (ABR)
5. Recherche dans un ABR
6. Insertion dans un ABR
7. Suppression dans un ABR
8. Hauteur et Complexité Globale
9. Analyse Probabiliste
10. ABR comme algorithme de tri
11. Introduction aux AVL
12. Conclusion

1.1 — Définition : Arbre

Définition

Un **arbre** est une structure de données **hiérarchique** composée de **nœuds** reliés par des **arêtes**, avec un nœud distingué appelé **racine**.



Racine :	A
Feuilles :	D, E, F, G
Hauteur :	2
Nœuds internes :	A, B, C

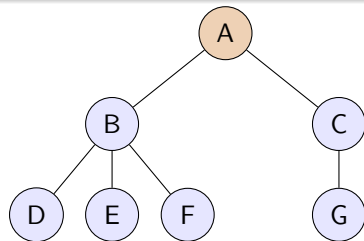
1.1 — Définition : Arbre

Définition

Un **arbre** est une structure de données **hiérarchique** composée de **nœuds** reliés par des **arêtes**, avec un nœud distingué appelé **racine**.

Propriétés fondamentales :

- ▶ Entre deux nœuds, il existe **exactement un chemin**.
- ▶ Chaque nœud (sauf la racine) possède **un seul parent**.
- ▶ Les nœuds sans fils sont appelés **feuilles**.



Vocabulaire :

- ▶ **Niveau** d'un nœud = distance depuis la racine
- ▶ **Hauteur** = niveau maximal de l'arbre

Racine :	A
Feuilles :	D, E, F, G
Hauteur :	2
Nœuds internes :	A, B, C

1.2 — Usages des arbres

Exemples concrets

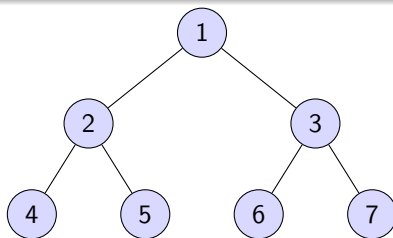
- ▶ **Système de fichiers** : arborescence de dossiers/fichiers
- ▶ **Arbres de jeux** : exploration de parties d'échecs
- ▶ **Expressions arithmétiques** : $(a + b) \times c$
- ▶ **HTML/DOM** : structure d'une page web
- ▶ **Tri et recherche** : ABR, AVL, tas, ...

2.1 — Définition : Arbre Binaire

Définition

Un **arbre binaire** est un arbre dans lequel chaque nœud possède **au plus deux fils** :

- ▶ un **fils gauche**
- ▶ un **fils droit**

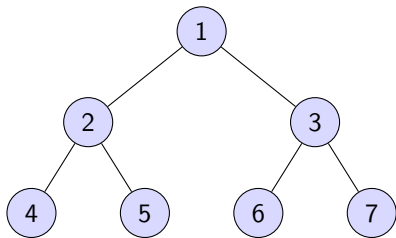


2.2 — Exemple : Arbre Binaire

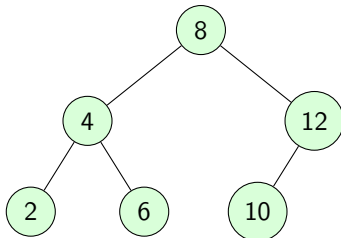
Cas particuliers

- ▶ **Arbre parfait** : feuilles aux deux derniers niveaux, rempli de gauche à droite
- ▶ **Arbre complet** : toutes les feuilles au même niveau
- ▶ **Arbre dégénéré** : chaque nœud a exactement un fils (= liste chaînée)

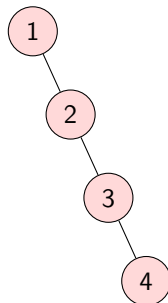
Arbre parfait



Arbre complet (équilibré)



Arbre dégénéré



2.4 — Théorème : Taille d'un arbre complet

Théorème

Un arbre binaire **complet** de hauteur h contient exactement :

$$N = 2^{h+1} - 1 \quad \text{nœuds}$$

Preuve par récurrence :

- ▶ *Base* ($h = 0$) : $N = 1$, et $2^1 - 1 = 1$
- ▶ *Hérédité* : un arbre de hauteur $h + 1 = 2$ sous-arbres de hauteur h + racine :

$$N = (2^{h+1} - 1) + (2^{h+1} - 1) + 1 = 2^{h+2} - 1$$

Conséquence

Pour N nœuds dans un arbre complet : $h = \lfloor \log_2 N \rfloor$

2.3 — Structure d'un nœud (C vs Java)

Structure Node en C

```
1 typedef struct Node {  
2     int key;  
3     struct Node* left;  
4     struct Node* right;  
5 } Node;
```

Création d'un nœud

```
1 Node* createNode(int key) {  
2     Node* n ;  
3     n=(Node*)malloc(sizeof(Node));  
4     n->key    = key;  
5     n->left   = NULL;  
6     n->right  = NULL;  
7     return n;  
8 }
```

2.3 — Structure d'un nœud (C vs Java)

Structure Node en C

```
1 typedef struct Node {  
2     int key;  
3     struct Node* left;  
4     struct Node* right;  
5 } Node;
```

Création d'un nœud

```
1 Node* createNode(int key) {  
2     Node* n ;  
3     n=(Node*)malloc(sizeof(Node));  
4     n->key    = key;  
5     n->left   = NULL;  
6     n->right  = NULL;  
7     return n;  
8 }
```

Classe Node en Java

```
1 class Node {  
2     int key;  
3     Node left;  Node right;  
4  
5     Node(int key) {  
6         this.key = key;  
7         this.left = null;  
8         this.right = null;  
9     }  
10 }
```

Création d'un nœud

```
1 Node n = new Node(10);
```

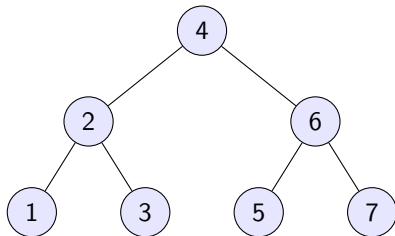
3.1 — Définition : Les Parcours

Définition

Un **parcours** est une façon de visiter **tous les nœuds** d'un arbre exactement une fois.

Nom	Ordre de visite	Méthode
InOrdre	Gauche → Racine → Droite	Récursif
PréOrdre	Racine → Gauche → Droite	Récursif
PostOrdre	Gauche → Droite → Racine	Récursif
LevelOrder	Niveau par niveau	File (BFS)

3.2 — Exemple : Les 4 parcours sur un même arbre



InOrdre : 1 → 2 → 3 → 4 → 5 → 6 → 7

PréOrdre : 4 → 2 → 1 → 3 → 6 → 5 → 7

PostOrdre : 1 → 3 → 2 → 5 → 7 → 6 → 4

LevelOrder : 4 → 2 → 6 → 1 → 3 → 5 → 7

3.3 — Algorithme : Parcours InOrdre

Pseudo-code

`inordre(nœud) :`

1. Si `nœud = NULL`, retourner
2. `inordre(nœud.gauche)`
3. Visiter `nœud`
4. `inordre(nœud.droite)`

3.3 — Algorithme : Parcours InOrdre

Pseudo-code

`inordre(nœud) :`

1. Si `nœud = NULL`, retourner
2. `inordre(nœud.gauche)`
3. Visiter `nœud`
4. `inordre(nœud.droite)`

Idée clé : Chaque nœud est traité *entre* ses deux sous-arbres.

Analyse

- ▶ Complexité **temps** : $O(N)$ — chaque nœud visité une fois
- ▶ Complexité **espace** : $O(h)$ — profondeur de la pile récursive

3.4 — Parcours InOrdre (C vs Java)

Code C

```
1  /* Parcours InOrdre : gauche,  
      racine, droite */  
2  void inorder(Node* root) {  
3      if (root != NULL) {  
4          inorder(root->left);  
5          printf("%d ",  
6              root->key);  
7          inorder(root->right);  
8      }  
9  }
```

3.4 — Parcours InOrdre (C vs Java)

Code C

```
1  /* Parcours InOrdre : gauche,  
   * racine, droite */  
2  void inorder(Node* root) {  
3      if (root != NULL) {  
4          inorder(root->left);  
5          printf("%d ",  
6              root->key);  
7          inorder(root->right);  
8      }  
9  }
```

Code Java

```
1  /* Parcours InOrdre : gauche, racine,  
   * droite */  
2  void inorder(Node root) {  
3      if (root != null) {  
4          inorder(root.left);  
5          System.out.print(root.key + " ");  
6          inorder(root.right);  
7      }  
8  }
```

4.1 — Définition : ABR

Définition

Un **Arbre Binaire de Recherche (ABR)** est un arbre binaire tel que pour **tout nœud** x :

$$\forall n \in \text{sous-arbre gauche}(x) : n.\text{clé} < x.\text{clé}$$

$$\forall n \in \text{sous-arbre droit}(x) : n.\text{clé} > x.\text{clé}$$

4.1 — Définition : ABR

Définition

Un **Arbre Binaire de Recherche (ABR)** est un arbre binaire tel que pour **tout nœud** x :

$$\forall n \in \text{sous-arbre gauche}(x) : n.\text{clé} < x.\text{clé}$$

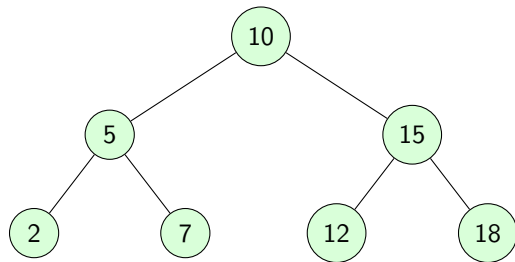
$$\forall n \in \text{sous-arbre droit}(x) : n.\text{clé} > x.\text{clé}$$

Propriété fondamentale

Le **parcours InOrdre** d'un ABR donne les clés **triées par ordre croissant**.

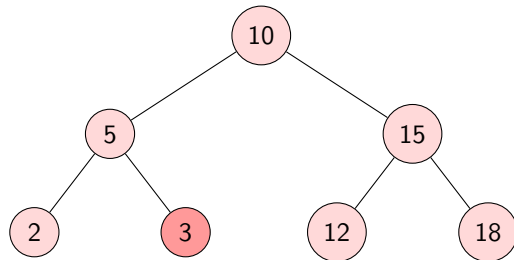
4.2 — Exemple : ABR valide vs non-valide

ABR valide



InOrdre : 2, 5, 7, 10, 12, 15, 18

Non-valide (3 < 5)



3 viole la règle : il doit être > 5

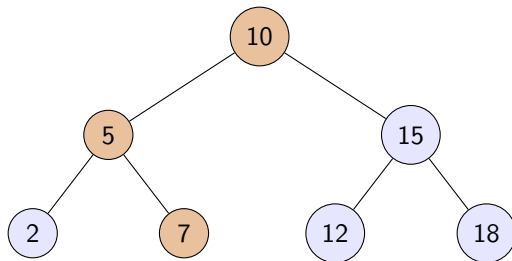
5.1 — Définition : Recherche

Définition

La **recherche** d'une clé k dans un ABR consiste à :

1. Comparer k avec la racine
2. Si k est plus petit $\rightarrow$ chercher dans le **sous-arbre gauche**
3. Si k est plus grand $\rightarrow$ chercher dans le **sous-arbre droit**
4. Si k est égal $\rightarrow$ **trouvé**
5. Si nœud NULL $\rightarrow$ **absent**

5.2 — Exemple : Recherche de 7



Chemin : $10 \rightarrow 5 \rightarrow 7$ (3 comparaisons)

5.3 — Algorithme : Recherche

Pseudo-code

`search(nœud, k) :`

1. Si `nœud = NULL` → retourner **faux**
2. Si `k = nœud.clé` → retourner **vrai**
3. Si `k < nœud.clé` → retourner `search(nœud.gauche, k)`
4. Sinon → retourner `search(nœud.droite, k)`

5.3 — Algorithme : Recherche

Pseudo-code

`search(nœud, k) :`

1. Si `nœud = NULL` → retourner **faux**
2. Si `k = nœud.clé` → retourner **vrai**
3. Si `k < nœud.clé` → retourner `search(nœud.gauche, k)`
4. Sinon → retourner `search(nœud.droite, k)`

Invariant : si k est dans l'arbre, il se trouve dans le sous-arbre courant.

Complexité de la Recherche

Cas	Situation	Complexité
Meilleur	Clé = racine	$O(1)$
Moyen	ABR équilibré aléatoire	$O(\log N)$
Pire	ABR dégénéré (liste)	$O(N)$

5.4 — Code C : Recherche

```
1  /* Recherche de key dans l'ABR enracine en root */
2  Node* search(Node* root, int key) {
3      /* Cas de base : arbre vide ou cle trouvee */
4      if (root == NULL || root->key == key)
5          return root;
6
7      /* Cle dans le sous-arbre gauche */
8      if (key < root->key)
9          return search(root->left, key);
10
11     /* Cle dans le sous-arbre droit */
12     return search(root->right, key);
13 }
```

5.5 — Recherche dans un ABR (Java)

Code Java

```
1  /* Recherche de key dans l'ABR enracine en root */
2  Node search(Node root, int key) {
3      /* Cas de base : arbre vide ou cle trouvee */
4      if (root == null || root.key == key)
5          return root;
6
7      /* Cle dans le sous-arbre gauche */
8      if (key < root.key)
9          return search(root.left, key);
10
11     /* Cle dans le sous-arbre droit */
12     return search(root.right, key);
13 }
```

6.1 — Définition : Insertion

Définition

L'**insertion** d'une clé k dans un ABR consiste à :

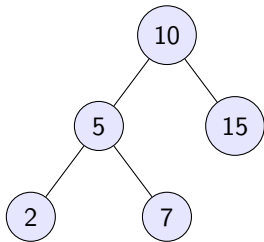
1. Rechercher la position où k devrait se trouver
2. Créer un nouveau nœud **feuille** à cette position

Invariant

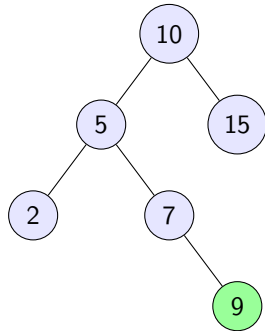
Après insertion, la propriété ABR (gauche $<$ racine $<$ droite) doit être **conservée**.

6.2 — Exemple : Insertion de 9

Avant insertion



Après insertion de 9



Chemin : $10 \rightarrow 5 \rightarrow 7 \rightarrow$ insérer à droite de 7

6.3 — Algorithme : Insertion

Pseudo-code

`insert(nœud, k) :`

1. Si `nœud = NULL` $\rightarrow$ retourner `créer_nœud(k)`
2. Si `k < nœud.clé` $\rightarrow$ `nœud.gauche = insert(nœud.gauche, k)`
3. Sinon $\rightarrow$ `nœud.droite = insert(nœud.droite, k)`
4. Retourner `nœud`

Remarque : on retourne le nœud à chaque étape pour reconstruire les liens parent-fils.

6.3 — Algorithme : Insertion

Pseudo-code

`insert(nœud, k) :`

1. Si `nœud = NULL` $\rightarrow$ retourner `créer_nœud(k)`
2. Si `k < nœud.clé` $\rightarrow$ `nœud.gauche = insert(nœud.gauche, k)`
3. Sinon $\rightarrow$ `nœud.droite = insert(nœud.droite, k)`
4. Retourner `nœud`

Remarque : on retourne le nœud à chaque étape pour reconstruire les liens parent-fils.

Complexité de l'Insertion

Cas	Situation	Complexité
Meilleur	Racine = NULL (arbre vide)	$O(1)$
Moyen	ABR équilibré	$O(\log N)$
Pire	ABR dégénéré	$O(N)$

6.4 — Code C : Insertion

```
1  /* Insere key dans l'ABR et retourne la nouvelle racine */
2  Node* insert(Node* root, int key) {
3      /* Position trouvee : creer le noeud feuille */
4      if (root == NULL)
5          return createNode(key);
6
7      /* Descendre a gauche */
8      if (key < root->key)
9          root->left = insert(root->left, key);
10     /* Descendre a droite */
11     else if (key > root->key)
12         root->right = insert(root->right, key);
13     /* Cle deja presente : ignorer (pas de doublon) */
14
15     return root; /* retourner la racine inchangee */
16 }
```

6.5 — Insertion dans un ABR (Java)

Code Java

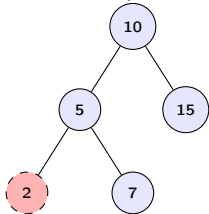
```
1  /* Insere key dans l'ABR et retourne la nouvelle racine */
2  Node insert(Node root, int key) {
3      /* Position trouvee : creer le noeud feuille */
4      if (root == null)
5          return new Node(key);
6
7      /* Descendre a gauche */
8      if (key < root.key)
9          root.left = insert(root.left, key);
10     /* Descendre a droite */
11     else if (key > root.key)
12         root.right = insert(root.right, key);
13     /* Cle deja presente : ignorer (pas de doublon) */
14
15     return root; /* retourner la racine inchangee */
16 }
```

7.1 — Définition : Suppression

Définition

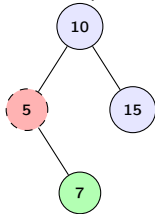
La **suppression** d'une clé k dans un ABR se traite en **3 cas** selon le nombre de fils du nœud à supprimer :

Cas 1 : Feuille (Supprimer 2)



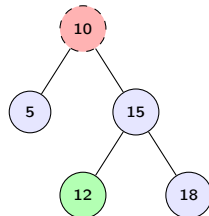
⇒ suppression directe

Cas 2 : 1 fils (Supprimer 5)



⇒ 7 remplace 5

Cas 3 : 2 fils (Supprimer 10)



⇒ 12 (min droit) remplace 10

Successeur

Le **successeur** d'un nœud = le **minimum du sous-arbre droit** (nœud le plus à gauche).

7.2 — Algorithme de Suppression et sa complexité

Pseudo-code

`delete(nœud, k) :`

1. Si `nœud = NULL` $\rightarrow$ retourner `NULL`
2. Si `k < nœud.clé` $\rightarrow$ `nœud.gauche = delete(gauche, k)`
3. Si `k > nœud.clé` $\rightarrow$ `nœud.droite = delete(droite, k)`
4. Sinon (clé trouvée) :
 - Cas 1/2 : si un fils `NULL` $\rightarrow$ retourner l'autre fils
 - Cas 3 : trouver `min(droite)`, copier sa clé, supprimer `min(droite)`
5. Retourner `nœud`

Cas	Situation	Complexité
Meilleur	Suppression d'une feuille proche racine	$O(1)$
Moyen	ABR équilibré	$O(\log N)$
Pire	ABR dégénéré	$O(N)$

7.3 — Code C : Trouver le minimum

```
1  /* Retourne le noeud avec la cle minimale */
2  Node* findMin(Node* root) {
3      /* Le minimum est le noeud le plus a gauche */
4      while (root->left != NULL)
5          root = root->left;
6      return root;
7  }
```

7.4 — Code C : Suppression

```
1 Node* delete(Node* root, int key) {
2     if (root == NULL) return NULL;
3
4     if (key < root->key)
5         root->left = delete(root->left, key);
6     else if (key > root->key)
7         root->right = delete(root->right, key);
8     else {
9         /* Cas 1 & 2 : 0 ou 1 fils */
10        if (root->left == NULL) return root->right;
11        if (root->right == NULL) return root->left;
12
13        /* Cas 3 : 2 fils - remplacer par successeur */
14        Node* succ = findMin(root->right);
15        root->key = succ->key;
16        root->right = delete(root->right, succ->key);
17    }
18    return root;
19 }
```

7.5 — Trouver le minimum (Java)

Code Java

```
1  /* Retourne le noeud avec la cle minimale */
2  Node findMin(Node root) {
3      /* Le minimum est le noeud le plus a gauche */
4      while (root.left != null)
5          root = root.left;
6      return root;
7  }
```

7.6 — Suppression dans un ABR (Java)

Code Java

```
1 Node delete(Node root, int key) {
2     if (root == null) return null;
3     if (key < root.key)
4         root.left = delete(root.left, key);
5     else if (key > root.key)
6         root.right = delete(root.right, key);
7     else {
8         /* Cas 1 & 2 : 0 ou 1 fils */
9         if (root.left == null) return root.right;
10        if (root.right == null) return root.left;
11        /* Cas 3 : 2 fils - remplacer par successeur */
12        Node succ = findMin(root.right);
13        root.key = succ.key;
14        root.right = delete(root.right, succ.key);
15    }
16    return root;
17 }
```

8.1 — Définition : Hauteur et son rôle

Définition

La **hauteur** h d'un ABR détermine la complexité de **toutes** les opérations :

$$T_{\text{opération}}(N) = O(h)$$

Théorème fondamental

Pour un ABR à N nœuds :

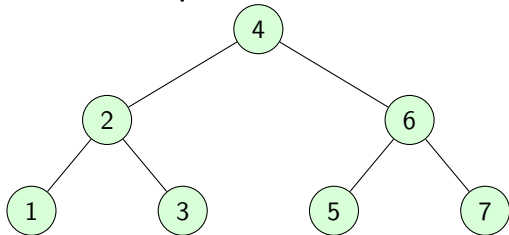
$$\log_2 N \leq h \leq N - 1$$

Cas extrêmes :

- ▶ Arbre **équilibré** : $h \approx \log_2 N$ (optimal)
- ▶ Arbre **dégénéré** : $h = N - 1$ (pire cas)

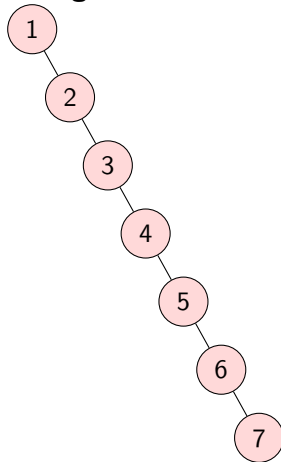
8.2 — Exemple : Équilibré vs Dégénéré ($N = 7$)

Équilibré : $h = 2$



Inséré dans l'ordre : 4,2,6,1,3,5,7

Dégénéré : $h = 6$



Inséré dans l'ordre : 1,2,3,4,5,6,7

8.3 — Preuve : Borne sur la hauteur

Borne inférieure ($h \geq \log_2 N$) :

- ▶ Un arbre de hauteur h contient **au plus** $2^{h+1} - 1$ nœuds
- ▶ Donc : $N \leq 2^{h+1} - 1 \Rightarrow N + 1 \leq 2^{h+1}$
- ▶ $\Rightarrow h + 1 \geq \log_2(N + 1) \Rightarrow \boxed{h \geq \log_2 N} \checkmark$

Borne supérieure ($h \leq N - 1$) :

- ▶ Dans le pire cas, chaque nœud a **exactement un fils** (chaîne)
- ▶ On a alors $h = N - 1 \checkmark$

8.4 — Analyse : Tableau récapitulatif

Complexité de toutes les opérations

Opération	Meilleur	Moyen	Pire
Recherche	$O(1)$	$O(\log N)$	$O(N)$
Insertion	$O(\log N)$	$O(\log N)$	$O(N)$
Suppression	$O(\log N)$	$O(\log N)$	$O(N)$
Min / Max	$O(\log N)$	$O(\log N)$	$O(N)$

Conclusion clé

Un ABR est **efficace** ssi il est **équilibré**. Le pire cas ($O(N)$) survient lors d'insertions ordonnées.

9.1 — Définition : Complexité moyenne

Définition

La **complexité moyenne** d'un ABR est calculée en supposant que les N clés sont insérées dans un **ordre aléatoire uniforme** (toutes les permutations équiprobables).

Objectif : calculer l'espérance du nombre de comparaisons lors d'une recherche.

9.2 — Algorithme d'analyse : Variables indicatrices

Méthode

Pour chaque paire (i, j) avec $i < j$, on définit :

$$X_{ij} = \begin{cases} 1 & \text{si } i \text{ et } j \text{ sont comparés} \\ 0 & \text{sinon} \end{cases}$$

Nombre total de comparaisons :

$$X = \sum_{i < j} X_{ij}$$

Par linéarité de l'espérance :

$$E[X] = \sum_{i < j} P(i \text{ comparé à } j)$$

9.3 — Probabilité de comparaison de deux clés dans un ABR

Idée clé

i et j sont comparés **si et seulement si** l'un d'eux est inséré en premier parmi tous les éléments de l'intervalle $[i, j]$.

- ▶ Si un élément $k \in [i, j]$ est inséré avant i et j , il devient la racine d'un sous-arbre contenant i et j .
- ▶ Alors i et j ne seront jamais comparés directement.
- ▶ Seul le premier élément de l'intervalle $[i, j]$ crée une comparaison directe entre i et j .

Intuition

- ▶ Plus i et j sont **proches**, plus la probabilité de comparaison est grande.
- ▶ Plus i et j sont **éloignés**, moins la probabilité de comparaison est grande.
- ▶ Cette formule est la base pour calculer le nombre moyen de comparaisons dans un ABR aléatoire.

9.4 — Calcul de la probabilité

Résultat clé

L'intervalle $[i, j]$ contient $j - i + 1$ éléments. Chaque élément a la même chance d'être inséré en premier :

$$P(i \text{ comparé à } j) = P(\text{premier élément} = i \text{ ou } j) = \frac{1}{j - i + 1} + \frac{1}{j - i + 1} = \frac{2}{j - i + 1}$$

Donc :

$$E[X] = \sum_{i < j} \frac{2}{j - i + 1} \approx 2N \ln N$$

4.9 Preuve de l'approximation

On veut simplifier la somme pour obtenir une formule pratique :

$$\sum_{i < j} \frac{1}{j - i + 1} = \sum_{i=1}^{N-1} \sum_{k=1}^{N-i} \frac{1}{k}$$

Ici, on fait le changement de variable $k = j - i + 1$, ce qui transforme la somme en une **somme harmonique**: $\sum_{k=1}^{N-i} \frac{1}{k}$ qui peut être approximée par le logarithme :

$$\sum_{k=1}^m \frac{1}{k} \approx \ln m + \gamma,$$

où $\gamma \approx 0.577$ est la constante d'Euler-Mascheroni (que l'on peut négliger pour une approximation grossière). Donc en sommant sur i , on obtient :

$$\sum_{i < j} \frac{2}{j - i + 1} \approx 2 \sum_{i=1}^N \ln \frac{N}{i} \approx 2N \ln N.$$

9.5 — Résultat final

Conversion en base 2 :

$$2 \ln N = 2 \ln 2 \cdot \log_2 N \approx 1.386 \cdot \log_2 N$$

Théorème — Complexité moyenne

La complexité moyenne d'une recherche dans un ABR aléatoire est :

$$T_{\text{moy}}(N) \approx 1.386 \cdot \log_2 N$$

Interprétation

- ▶ Un ABR aléatoire est **presque aussi bon** qu'un arbre parfaitement équilibré
- ▶ Mais le pire cas reste $O(N)$ — d'où l'intérêt des AVL

10.1 — Tri par ABR (Définition)

Propriété

Le **parcours InOrdre** d'un ABR produit les éléments dans l'ordre **croissant** — on peut donc trier avec un ABR.

Algorithme :

1. Insérer les N éléments un à un dans un ABR : $O(N \log N)$
2. Effectuer le parcours InOrdre : $O(N)$

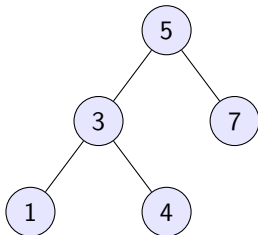
$$T_{\text{tri par ABR}}(N) = O(N \log N)$$

Attention

Si les éléments sont déjà triés à l'insertion → arbre dégénéré → $O(N^2)$!

10.2 — Exemple : Tri de {5, 3, 7, 1, 4}

ABR construit



InOrdre :

$1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7$

= Suite triée !

11.1 — Arbre AVL (Définition)

Définition

Un arbre **AVL** (Adelson-Velsky & Landis, 1962) est un ABR tel que pour **chaque nœud** :

$$|h_{\text{gauche}} - h_{\text{droite}}| \leq 1$$

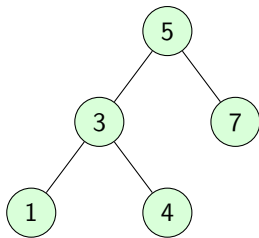
où h_{gauche} et h_{droite} sont les hauteurs des deux sous-arbres.

Pourquoi les AVL ?

Un ABR quelconque peut dégénérer en liste ($h = N - 1$). Un AVL garantit toujours $h = O(\log N)$.

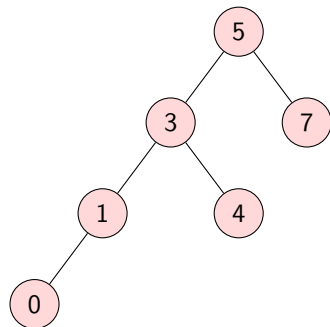
11.2 — Exemple : AVL valide vs non-valide

AVL valide



Tous les $|\Delta h| \leq 1$

Non-AVL



$|h_g - h_d| = 2$ à la racine ×

11.3 — Algorithme : Rotations AVL

Principe

Quand la condition AVL est violée après une insertion ou suppression, on restaure l'équilibre via des **rotations** (opérations en $O(1)$).

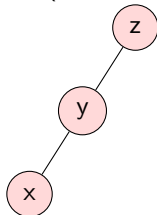
Cas	Rotation
LL (déséquilibre gauche-gauche)	Rotation droite
RR (déséquilibre droite-droite)	Rotation gauche
LR (déséquilibre gauche-droite)	Rotation gauche puis droite
RL (déséquilibre droite-gauche)	Rotation droite puis gauche

11.4 — Exemple : Rotation LL

Cas LL — déséquilibre à gauche-gauche

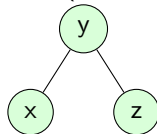
Insertion dans le sous-arbre gauche du fils gauche $\rightarrow$ rotation droite sur x .

Avant (déséquilibré)



rot. droite $\rightarrow$

Après (équilibré)



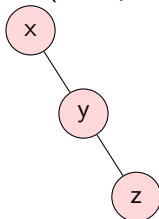
Preuve : L'ordre est conservé car $x < y < z$ avant et après.

11.4b — Exemple : Rotation RR

Cas RR — déséquilibre à droite-droite

Insertion dans le sous-arbre **droit** du **fil droit** → **rotation gauche** sur x .

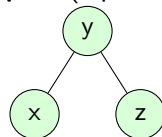
Avant (déséquilibré)



$$\text{bal}(x) = -2$$

rot. gauche sur x →

Après (équilibré)



$$\text{bal}(y) = 0$$

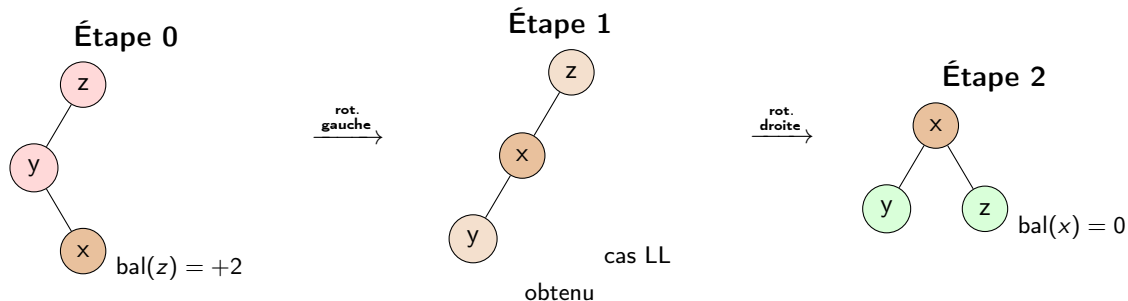
Invariant : $x < y < z$ avant et après la rotation.

Symétrique de la rotation LL (miroir gauche/droite).

11.4c — Exemple : Rotation LR

Cas LR — déséquilibre gauche-droite

Insertion dans le sous-arbre **droit** du fils **gauche** → **double rotation** : gauche sur y, puis droite sur z.

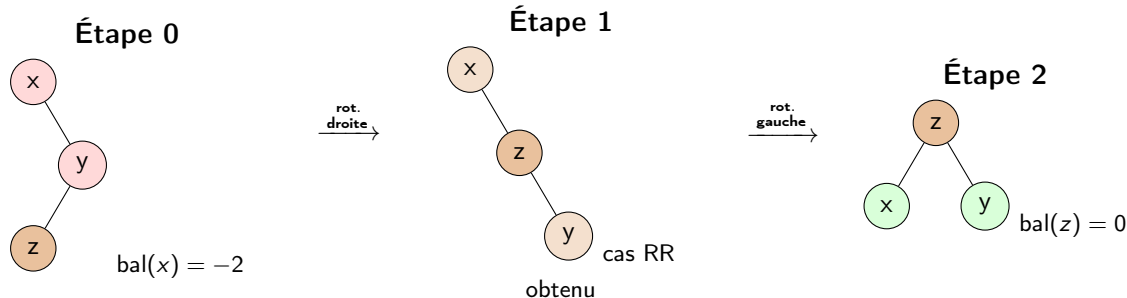


Invariant : $y < x < z$ est conservé à chaque étape.

11.4d — Exemple : Rotation RL

Cas RL — déséquilibre droite-gauche

Insertion dans le sous-arbre gauche du fils droit → double rotation : droite sur y , puis gauche sur x .



Invariant : $x < z < y$ est conservé à chaque étape.

Symétrique de la rotation LR (miroir gauche/droite).

11.4e — Synthèse : Les 4 cas de rotation AVL

Cas	Déséquilibre	Rotation(s)	Résultat
LL	gauche-gauche	1 rotation droite	nœud du milieu monte
RR	droite-droite	1 rotation gauche	nœud du milieu monte
LR	gauche-droite	rot. gauche + droite	nœud interne monte
RL	droite-gauche	rot. droite + gauche	nœud interne monte

Invariant commun à toutes les rotations

Les rotations **conservent la propriété ABR** (ordre infixe inchangé) tout en **réduisant la hauteur** locale de 1.

Coût : chaque rotation est $O(1)$ — on ne fait que modifier quelques pointeurs.

11.5 — Analyse : Hauteur d'un AVL

Théorème

Un arbre AVL à N nœuds a une hauteur :

$$h \leq 1.44 \cdot \log_2 N$$

Idée de la preuve :

- ▶ Le nombre minimum de nœuds d'un AVL de hauteur h suit la récurrence de Fibonacci :

$$N(h) = 1 + N(h-1) + N(h-2)$$

- ▶ $N(h) \geq \phi^h$ où $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (nombre d'or)
- ▶ Donc : $h \leq \log_{\phi} N = \frac{\log_2 N}{\log_2 \phi} \approx 1.44 \log_2 N$

11.6 — Définition : Arbre Rouge-Noir (Red-Black Tree)

Définition

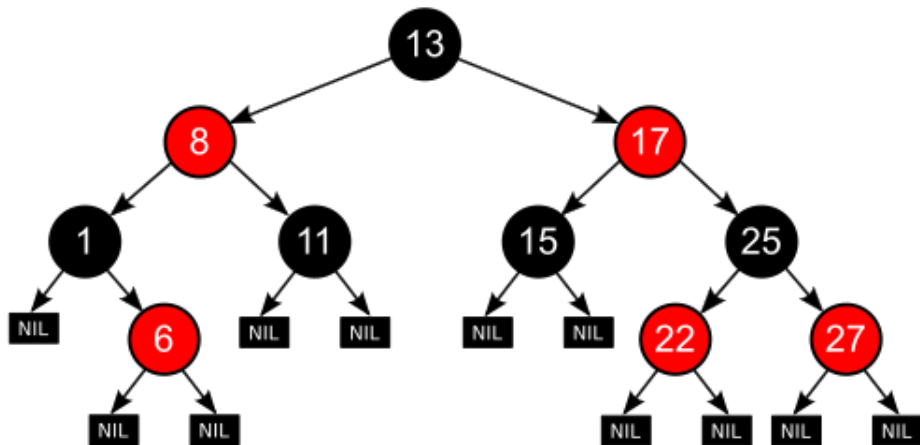
Un **arbre Rouge-Noir** est un ABR dans lequel chaque nœud est colorié **rouge** ou **noir**, et qui respecte les 5 propriétés suivantes :

1. Tout nœud est **rouge** ou **noir**
2. La **racine** est toujours **noire**
3. Toutes les **feuilles** (NIL sentinelles) sont **noires**
4. Un nœud **rouge** ne peut avoir que des fils **noirs** (pas deux rouges consécutifs)
5. Pour tout nœud, tous les chemins vers les feuilles contiennent le **même nombre de nœuds noirs** (hauteur noire bh)

Objectif

Ces propriétés garantissent un équilibre **approximatif** — suffisant pour assurer $h = O(\log N)$.

11.7 — Exemple : Arbre Rouge-Noir



Hauteur noire $bh = 2$ sur tous les chemins

Hauteur totale $h = 4$

11.8 — Algorithme : Maintien des propriétés

Après insertion ou suppression

Un Red-Black Tree se rééquilibre via deux types d'opérations :

1. **Recoloration** (changement de couleur) :

- ▶ Changer la couleur d'un nœud : $O(1)$
- ▶ Peut se propager vers le haut (jusqu'à la racine)

2. **Rotations** (comme les AVL) :

- ▶ Rotation gauche ou droite : $O(1)$
- ▶ Au maximum **2 rotations** par insertion (vs potentiellement $O(\log N)$ pour AVL)
- ▶ Au maximum **3 rotations** par suppression

Avantage clé sur AVL

Moins de rotations en pratique → Red-Black préféré dans les implémentations système (C++ `std::map`, Java `TreeMap`, Linux kernel).

11.9 — Théorème : Hauteur d'un Red-Black Tree

Théorème

Un arbre Rouge-Noir à N nœuds internes a une hauteur :

$$h \leq 2 \log_2(N + 1)$$

Idée de la preuve :

- ▶ Soit bh la **hauteur noire** (nombre de nœuds noirs sur tout chemin)
- ▶ Propriété 4 ($\Rightarrow$ pas deux rouges consécutifs) implique : $bh \geq \frac{h}{2}$
- ▶ Le sous-arbre enraciné en x contient au moins $2^{bh} - 1$ nœuds internes
- ▶ Donc : $N \geq 2^{h/2} - 1$
- ▶ En inversant :

$$h \leq 2 \log_2(N + 1)$$

Conséquence

Toutes les opérations (recherche, insertion, suppression) sont en $O(\log N)$ **garanti**.

11.10 — Analyse : Comparaison ABR vs AVL vs Red-Black

Résumé des hauteurs garanties

Structure	Hauteur	Garantie ?	Rotations/op
ABR simple	$\leq N - 1$	×	0
AVL	$\leq 1.44 \log_2 N$	✓	$O(\log N)$
Red-Black	$\leq 2 \log_2 N$	✓	≤ 3

Comparaison AVL vs Red-Black

- ▶ **AVL** : plus équilibré (1.44 vs 2) → recherche légèrement plus rapide
- ▶ **Red-Black** : moins de rotations → insertions/suppressions plus rapides
- ▶ **En pratique** : Red-Black dominant dans les bibliothèques standard

11.11 — Analyse : Comparaison ABR vs AVL

Tableau comparatif

	ABR	AVL	Red-Black
Hauteur max	$N - 1$	$1.44 \log_2 N$	$2 \log_2 N$
Recherche	$O(N)$ pire	$O(\log N)$ garanti	$O(\log N)$ garanti
Insertion	$O(N)$ pire	$O(\log N) + \text{rot.}$	$O(\log N) + \text{rot.}$
Complexité	Rotations ?	Oui (AVL)	Moins

Conclusion pratique

- ▶ **ABR** : simple, bon en moyenne, mauvais pire cas
- ▶ **AVL** : optimal en recherche, rotations à chaque opération
- ▶ **Red-Black** : compromis — moins de rotations, garanties maintenues

Conclusion — Synthèse du cours

Ce qu'il faut retenir

1. Un **ABR** organise les données pour une recherche efficace : $O(\log N)$ en moyenne
2. Toutes les opérations (recherche, insertion, suppression) ont la même complexité : $O(h)$
3. La **hauteur** h est le facteur critique : $\log_2 N \leq h \leq N - 1$
4. Un ABR dégénère en liste si les données arrivent dans l'ordre trié
5. Les **arbres équilibrés** (AVL, Red-Black) garantissent $h = O(\log N)$

À retenir absolument

Un ABR est un excellent outil **si et seulement s'il reste équilibré**. C'est pourquoi les structures auto-équilibrantes (AVL, Red-Black, B-arbres) sont indispensables en pratique.